PROGRAMACIÓN EN AMBIENTE WEB 2 · SEXTO SEMESTRE

Nueve sistemas, una misma arquitectura

Cada equipo construye un sistema distribuido real: frontend en Angular, backend en microservicios Spring Boot y despliegue orquestado. Lo que cambia entre equipos es el problema que resuelven, y ese problema lo aporta otra materia que ya están cursando. El diseño no se improvisa: se deriva de atributos de calidad, con el método ADD.

La plataforma es la misma para todos

El equipo no puede elegir el stack. Esa decisión ya está tomada para que la discusión sea de arquitectura y no de herramientas, y para que los nueve proyectos se puedan comparar entre sí.

FRONTEND

Angular con componentes standalone y signals. Consume la API por HTTP; no habla con la base de datos ni contiene lógica de negocio.

BACKEND

Spring Boot, mínimo tres microservicios con responsabilidades separadas y su propia base de datos cuando aplique.

ORQUESTACIÓN

Kubernetes o **Docker Swarm**. Todo se levanta con un manifiesto; nadie arranca servicios a mano.

IDENTIDAD

Keycloak con **OAuth 2.0** y **OIDC**. Ningún endpoint queda abierto: la identidad se delega en un proveedor externo que emite tokens firmados, y cada servicio los valida por su cuenta.

CÓMPUTO NATIVO

Una **biblioteca compartida** en C dentro del contenedor, invocada desde Java con **JNA**. Obligatoria en los siete proyectos que tienen equipo de Arquitectura.

Autenticación y autorización, explicadas una vez

Este bloque es común a los nueve proyectos y es obligatorio. La matriz concreta de roles y operaciones está en el documento de cada proyecto; lo que sigue es el porqué y el mínimo que se exige en todos. **Corresponde únicamente a Programación en Ambiente Web 2: Arquitectura de Computadores no lo evalúa.**

Escribir un inicio de sesión propio —una tabla de usuarios, una contraseña con hash, una cookie de sesión— es más corto de programar que integrar un proveedor externo, y por eso es la salida habitual. También es la que no sobrevive al primer microservicio adicional: la sesión vive en un proceso y el segundo servicio no la conoce, de modo que o se comparte el almacenamiento de sesiones —y los servicios dejan de ser independientes— o cada uno vuelve a pedir credenciales.

Un token firmado resuelve exactamente eso. Keycloak emite un **JWT** firmado con su llave privada; cada servicio verifica la firma contra la llave pública publicada por el realm y obtiene la identidad y los roles *sin consultar a nadie* y sin guardar estado. Esa es la razón arquitectural, y es la que debe quedar escrita en la bitácora.

REALM PROPIO Un contenedor de Keycloak levantado por el mismo manifiesto, con un realm importado desde un archivo versionado: sus roles, sus clientes y al menos un usuario de prueba por rol.	**ANGULAR** Cliente **público** con Authorization Code y **PKCE**. Sin *secret* en el navegador. El token se adjunta con un interceptor; el frontend no decide permisos.
CADA SERVICIO *Resource server* de OAuth 2.0: verifica la firma contra el JWKS del realm y autoriza por rol sobre el endpoint. Sin token, 401. Con rol equivocado, 403.	**DOS ROLES MÍNIMO** Cada proyecto define al menos dos roles de aplicación con operaciones distintas, porque son los que permiten demostrar un rechazo por rol. El administrador de Keycloak no cuenta como rol de aplicación.

> **El error que más tiempo consume.** Keycloak coloca los roles del realm **anidados** dentro del *claim* `realm_access`, en la forma `realm_access.roles`. La configuración más citada de Spring Security para extraer autoridades busca el *claim* de forma **plana**, en el primer nivel del token, y por lo tanto no encuentra nada: el usuario queda autenticado pero sin autoridades, y todo responde 403 aunque el rol esté correctamente asignado. Las salidas son escribir un convertidor propio que descienda al *claim* anidado, o declarar en Keycloak un *mapper* que publique los roles en el primer nivel. La que se elija se anota en la bitácora.

La evidencia son cuatro respuestas HTTP, obtenidas contra el servicio y no contra el frontend: sin cabecera de autorización, 401; con un token cuya firma fue alterada, 401; con un token válido del rol que no corresponde, 403; con el token del rol correcto, 200. Las cuatro se entregan como registro reproducible, con el comando ejecutado junto a cada una.

El componente nativo, explicado una vez

Este es el aporte del curso de **Arquitectura de Computadores**, y es lo que distingue a estos proyectos de una aplicación web cualquiera. El algoritmo que resuelve el problema de la materia acompañante **no se escribe en Java**: se escribe en C, se compila como biblioteca compartida de Linux (`.so`), se copia dentro de la imagen del contenedor, y un servicio Spring Boot la invoca con JNA.

```
algoritmo.c  →  gcc -shared -fPIC  →  libalgo.so  →  imagen Docker  →  inte
```

JNA existe precisamente para no tener que escribir código puente: declaras una interfaz Java que describe la firma de la función en C y la biblioteca queda disponible como si fuera un objeto más. No hay que compilar nada intermedio.

Lo que se evalúa aquí no es que funcione, sino que sepas por qué está ahí: qué gana el sistema al bajar ese cálculo a código nativo, qué cuesta en complejidad de despliegue, y cómo cambia el tamaño de la imagen y el arranque del contenedor. Un equipo que no pueda justificar la decisión con una medición no cumple el requisito, aunque compile.

Cómo se reparte entre los dos cursos

Los equipos de Web 2 y los de Arquitectura de Computadores no coinciden: de los nueve, solo dos calzan exactamente. En lugar de reformar los grupos, el proyecto se reparte por **entregable**. Cada frente responde por algo distinto y la frontera entre los dos es una interfaz, no una reunión.

WEB 2 RESPONDE POR	ARQUITECTURA RESPONDE POR
El frontend en Angular, los microservicios Spring Boot, el modelo de datos, **la autenticación y la autorización**, los tres escenarios de calidad con su medición, y la bitácora de decisiones.	La biblioteca compartida en C, la interfaz JNA, el servicio que la expone, la medición de C contra Java, y el **apoyo al despliegue de la aplicación a producción**.

> **La autenticación y la autorización no se exigen en Arquitectura de Computadores.** Ese bloque pertenece por completo a Web 2. El servicio que expone la biblioteca nativa queda protegido igual que los demás, pero configurarlo y demostrarlo es responsabilidad del frente de Web 2.

QUÉ ES EL APOYO AL DESPLIEGUE

Sale naturalmente de quien construye el componente nativo, porque es quien tiene que incorporar una biblioteca compilada a una imagen. Son cuatro entregables concretos: el `Dockerfile` multietapa que compila el `.so` y produce una imagen final sin compilador; el manifiesto de despliegue de *todo* el sistema con los límites de CPU y memoria declarados; el arranque desde cero comprobado en una máquina limpia con un solo comando; y el tamaño de imagen y el tiempo de arranque medidos **antes y después** de incorporar la biblioteca. Bajar un cálculo a C cuesta algo en el despliegue, y ese costo se reporta con números.

EL CONTRATO, Y POR QUÉ LA DEPENDENCIA ES SEGURA

Lo que une los dos frentes se acuerda en la primera semana y se congela: la interfaz JNA exacta —nombres, tipos, y quién libera la memoria—, un conjunto de casos de referencia con su valor esperado escritos antes de implementar, y cómo se reporta un error desde C, que no tiene excepciones. Queda en la bitácora de ambos.

> **La aplicación debe funcionar sin la biblioteca nativa.** Todos los proyectos exigen ya una implementación en Java de la misma función, como línea base para medir si bajar a C valió la pena. Esa línea base es también el *respaldo*: el servicio escoge entre la ruta nativa y la de Java por configuración, de modo que si el frente nativo no está listo la aplicación sigue en pie y la evaluación de Web 2 no queda a merced de otro equipo. No es trabajo extra —ya estaba pedido para la medición—, pero sí una decisión de diseño que debe quedar escrita.

Qué se entrega del diseño

El diseño se hace con **ADD** (Attribute-Driven Design): la arquitectura no sale de dibujar cajas, sale de decidir qué atributos de calidad debe cumplir el sistema y de justificar cada decisión contra ellos. Estos cinco pasos son el entregable de diseño.

> 01
>
> **Drivers**
>
> El propósito del sistema, los requisitos funcionales principales, las restricciones que no se negocian y los atributos de calidad que importan.

02

Escenarios de atributos de calidad

Cada atributo escrito como escenario medible: quién lo provoca, qué hace el sistema y con qué número se comprueba. "Debe ser rápido" no es un escenario; "responde en menos de 300 ms con 500 registros" sí.

03

Descomposición

Qué microservicios existen, de qué responde cada uno y por qué el corte quedó ahí y no en otro lado.

04

Interfaces

El contrato de cada servicio: endpoints, formato de datos y qué pasa cuando el de al lado no responde.

05

Bitácora de decisiones

Cada decisión con las alternativas que se descartaron y contra qué escenario se justificó. Es lo que más pesa: una decisión sin alternativa evaluada es una decisión que no se tomó.

El vocabulario de los atributos de calidad

Un escenario mal nombrado es un escenario que no se puede discutir. Estos son los nombres estándar —los de **Len Bass, *Software Architecture in Practice***— y son los únicos que se aceptan en la entrega: nada de "precisión", "robustez" o "eficiencia" a secas.

ATRIBUTO	LA PREGUNTA QUE RESPONDE	SE MIDE EN
Disponibilidad	¿Qué pasa con el resto del sistema cuando una parte falla?	% de operación, tiempo de recuperación
Facilidad de despliegue	¿Con cuánto esfuerzo y riesgo llega una versión nueva a producción?	tiempo y pasos hasta desplegar
Eficiencia energética	¿Cuánta energía consume el sistema por unidad de trabajo?	julios o vatios por operación

ATRIBUTO	LA PREGUNTA QUE RESPONDE	SE MIDE EN
Integrabilidad	¿Cuánto cuesta hacer que este componente funcione con otro?	interfaces y adaptadores que hay que escribir
Modificabilidad	¿Cuánto cuesta agregar o cambiar una funcionalidad?	módulos que hay que tocar
Desempeño	¿Cuánto tarda en responder, y cómo cambia eso cuando la carga o los datos crecen?	ms, peticiones por segundo, curva tiempo vs. volumen
Seguridad	¿Quién puede hacer qué, y cómo se comprueba?	intentos rechazados, superficie expuesta
Seguridad física	¿Puede el sistema causar daño al fallar?	estados peligrosos evitados
Testabilidad	¿Con cuánta facilidad puede el sistema demostrar que no se equivoca?	tiempo de la suite, casos con valor esperado
Usabilidad	¿La persona entiende qué pasó y qué hacer?	pasos para completar, errores explicados

Tres nombres de uso frecuente no están en esa tabla, y conviene saber por qué. *Escalabilidad* no es un atributo aparte en Bass: el comportamiento cuando la carga o los datos crecen se escribe como un escenario de **Desempeño**, con el volumen puesto en el estímulo y el crecimiento del tiempo en la medida. *Eficiencia de recursos* tampoco: el consumo de memoria, CPU o disco va en el entorno y en la medida de un escenario de **Desempeño**. Y que el resultado sea el correcto es un asunto **funcional**, no un atributo de calidad —Bass los separa a propósito, porque un sistema puede ser correcto y aun así imposible de mantener—: lo arquitectural es con cuánta facilidad el sistema puede *demostrar* que no se equivoca, y eso se llama **Testabilidad**.

A cada proyecto se le asignó un atributo, y es el que aparece en su ficha. No es el único que debe cumplirse: es el que se exige demostrado con una medición en la sustentación. Además de ese, cada proyecto escribe **dos escenarios más** con atributos de esta tabla, elegidos y justificados. Dos proyectos pueden compartir atributo —el vocabulario de Bass es corto a propósito—: lo que los distingue no es el nombre, sino el escenario de seis partes y la medida con que se comprueba.

Los proyectos

El reparto no es arbitrario: cada equipo recibe el problema de la materia que **más integrantes comparten**, según el formulario de registro. Donde el equipo de Web 2 coincide con el de Arquitectura, el proyecto es uno solo para las dos materias.

FULL-STACK WEB `7 integrantes` `↔ Low level software · 5 en común`

Banco de métodos numéricos

Análisis numérico · 5 de 6 integrantes

Una plataforma donde se define una función, se ejecutan varios métodos de aproximación sobre ella y se comparan lado a lado: cuántas iteraciones necesitó cada uno, cómo cayó el error y cuál no convergió. Responde con datos una pregunta del curso que hoy se contesta a mano en el cuaderno.

MICROSERVICIOS

- **`funcion-service`** — funciones, intervalos y tolerancias
- **`calculo-service`** — invoca la biblioteca nativa
- **`historial-service`** — corridas y sus iteraciones
- **`api-gateway`** — punto de entrada único

FUNCIONALIDAD PRINCIPAL

- Registrar una función y su intervalo de trabajo
- Ejecutar bisección, Newton-Raphson y secante sobre ella
- Ver la tabla de iteraciones con el error de cada paso
- Comparar dos métodos sobre la misma función
- Detectar y reportar los casos que no convergen

> BIBLIOTECA NATIVA — LIBNUMERICO.SO
>
> Los tres métodos iterativos en C, devolviendo el arreglo completo de iteraciones en una sola llamada. El punto arquitectural: una función que itera miles de veces es exactamente el caso donde cruzar la frontera Java↔C una vez por corrida —y no una vez por iteración— cambia el resultado. Deben medirse las dos formas.

ATRIBUTO DE CALIDAD — DESEMPEÑO

Un docente lanza una corrida de Newton-Raphson con tolerancia 1e-10 sobre un polinomio de grado 5. El sistema devuelve la tabla completa de iteraciones en menos de **500 ms**, y la misma corrida resuelta en Java puro sirve de línea base para reportar la diferencia.

Dylan Steven Bedón Ramírez · Jonatan David Montoya Posada · Valentina Rosero Ocoro · Liseth Johanna Henao Cardona · Juan David Torres Vásquez · Cristóbal Moreno Montoya · Brian Campo Villada

INDEX LOGIC `4 integrantes` `↔ Index logic · 3 en común`

Simulador de jerarquía de memoria

Arquitectura de computadores · 3 de 3 integrantes

Una herramienta que recibe una traza de accesos a memoria, la corre contra distintas configuraciones de caché y muestra por qué el mismo programa tarda diferente según cómo recorra los datos. Es el único equipo cuya materia acompañante es la propia Arquitectura, así que aquí la biblioteca nativa no es un accesorio: es el sistema.

MICROSERVICIOS

- **`traza-service`** — trazas de acceso, subidas o generadas
- **`cache-service`** — invoca la biblioteca nativa
- **`comparador-service`** — corridas y métricas comparadas

FUNCIONALIDAD PRINCIPAL

- Cargar o generar una traza de direcciones
- Configurar tamaño de línea, vías y política de reemplazo
- Obtener aciertos, fallos y tasa de acierto
- Comparar mapeo directo contra asociativo por conjuntos
- Mostrar por qué recorrer una matriz por filas o por columnas cambia el resultado

BIBLIOTECA NATIVA — LIBCACHE.SO

Simulación de caché en C: mapeo directo y asociativa por conjuntos, con reemplazo LRU y FIFO. Recibe la traza completa y devuelve el conteo de aciertos y fallos. Aquí el argumento a favor del código nativo no es solo velocidad: es que el modelo de memoria del problema se expresa de forma natural con punteros y aritmética de direcciones.

ATRIBUTO DE CALIDAD — DESEMPEÑO

La traza de entrada pasa de diez mil a un millón de accesos, y se piden tres configuraciones de caché a la vez. El tiempo de respuesta crece de forma **lineal con el tamaño de la traza, no exponencial**, y el millón de accesos se resuelve en menos de **3 segundos**. Se mide con tres tamaños y se muestra la curva: si se dispara, el cuello de botella está en el diseño, no en el hardware.

Leidy Manuela González Marín · José David González González · Juan José Sánchez Gálvez · Miguel Ángel Martínez Peña

LOS NÁUFRAGOS 2 integrantes ↔ Los Náufragos · equipo idéntico

Planificador de procesos

Sistemas operativos · 2 de 2 integrantes

Un simulador que recibe una lista de procesos con sus tiempos y aplica distintos algoritmos de planificación, mostrando el diagrama de Gantt y las métricas de cada uno. Es el equipo con mejor coincidencia de materias de todo el curso —comparten cuatro— pero solo son dos personas, así que el alcance está deliberadamente acotado.

MICROSERVICIOS

- **`proceso-service`** — procesos y conjuntos de prueba
- **`planificador-service`** — invoca la biblioteca nativa

FUNCIONALIDAD PRINCIPAL

- Definir procesos con llegada, ráfaga y prioridad
- Ejecutar FIFO, SJF, Round Robin y prioridades

- Ver el diagrama de Gantt resultante
- Comparar tiempos de espera y retorno entre algoritmos

> BIBLIOTECA NATIVA — LIBPLANIFICADOR.SO
> Los cuatro algoritmos en C, devolviendo la línea de tiempo como arreglo de rebanadas más las métricas agregadas. Con dos integrantes, dos servicios y una biblioteca bien hecha valen más que cinco servicios a medias.

> ATRIBUTO DE CALIDAD — MODIFICABILIDAD
> **El equipo decide agregar un quinto algoritmo de planificación.** Solo se toca la biblioteca en C y se recompila la imagen: **ni el frontend ni el contrato de la API cambian**. Si agregar un algoritmo obliga a tocar Angular, el corte de responsabilidades quedó mal.

Kamila Marín Cossio · Juan Fernando Muñoz Silva

UCEVA DEVS `5 integrantes` `↔ Core Systems · 1 en común`

Presupuestador de obra

Costos y presupuesto · 3 de 3 integrantes

Una herramienta para armar el presupuesto de un trabajo: se eligen insumos, se indican cantidades y el sistema calcula qué cuesta, cuánto hay que cobrar y a partir de qué volumen deja de perder dinero. Es el único equipo donde los tres integrantes comparten la misma materia acompañante.

MICROSERVICIOS

- **`insumo-service`** — catálogo con precios y unidades
- **`presupuesto-service`** — presupuestos y sus ítems
- **`calculo-service`** — invoca la biblioteca nativa

FUNCIONALIDAD PRINCIPAL

- Mantener el catálogo de insumos
- Armar un presupuesto por capítulos

- Calcular costo directo, AIU y precio de venta
- Hallar el punto de equilibrio
- Comparar dos escenarios de precios

BIBLIOTECA NATIVA — LIBFINANZAS.SO

Cálculo financiero en C: amortización, valor presente neto y tasa interna de retorno por aproximaciones sucesivas. La TIR es un caso interesante porque no tiene fórmula cerrada: se resuelve iterando, igual que los métodos del otro equipo.

ATRIBUTO DE CALIDAD — TESTABILIDAD

Un usuario arma un presupuesto de 200 ítems con precios de tres decimales. El total calculado **no difiere en un solo peso** del mismo cálculo hecho en una hoja de cálculo de control. Debe definirse desde el diseño cómo se representa el dinero: el error de redondeo con punto flotante es aquí un problema real, no teórico.

Dos integrantes de Core Systems no cursan Web 2. Su aporte es la biblioteca nativa y su justificación arquitectural; no se les puede exigir el frontend ni el despliegue. La frontera de responsabilidad se define por escrito desde la primera semana.

Andrés Felipe Vargas Ceballos · Esteban Lozano Jaramillo · Juan David González Ospina · Diego Alejandro Laverde Restrepo · Luis Alejandro Pino Bellorín

VILLANOS `2 integrantes` `los dos frentes, mismo equipo`

Red de rutas

Matemáticas discretas · tema asignado

Un sistema para modelar la red de transporte de una ciudad como un grafo y responder preguntas sobre ella: la ruta más corta entre dos paradas, qué queda incomunicado si se cierra un tramo, si hay ciclos. El grafo se dibuja en pantalla y el recorrido del algoritmo se reproduce paso a paso.

MICROSERVICIOS

- **`grafo-service`** — paradas y tramos
- **`ruta-service`** — algoritmos de recorrido y camino mínimo
- **`api-gateway`** — punto de entrada único

FUNCIONALIDAD PRINCIPAL

- Definir la red de paradas y tramos con sus pesos
- Calcular la ruta más corta entre dos puntos
- Detectar componentes desconectadas y ciclos
- Reproducir el recorrido del algoritmo paso a paso

ATRIBUTO DE CALIDAD — DISPONIBILIDAD

El servicio de rutas se cae mientras un usuario está consultando la red. El sistema **sigue mostrando el grafo** y avisa que el cálculo de rutas no está disponible, en vez de quedarse en blanco. El frontend nunca debe depender de que todos los servicios estén vivos.

Las mismas dos personas responden por los dos frentes. Es el segundo equipo que coincide exactamente entre los dos cursos, junto con Los Náufragos, así que no hay contrato que negociar con nadie. `libgrafo.so` con Dijkstra es **obligatoria**, y se evalúa desde Arquitectura de Computadores.

Jhonatan Varón Henao · Santiago Ríos Morales

NOVADEVS `2 integrantes` `sin frente de Arquitectura`

Detector de similitud entre entregas

Matemáticas discretas · tema asignado

Un sistema que recibe las entregas escritas de un curso, las compara todas contra todas y señala las parejas sospechosamente parecidas, mostrando en qué se parecen. La medida de fondo es la distancia de edición: cuántos cambios hacen falta para convertir un texto en el otro, calculada con programación dinámica.

MICROSERVICIOS

- **`entregas-service`** — cursos, entregas y su normalización
- **`similitud-service`** — comparación e informes
- **`api-gateway`** — punto de entrada único

FUNCIONALIDAD PRINCIPAL

- Cargar las entregas de un curso y normalizarlas
- Comparar todas contra todas y ordenar por parecido
- Filtrar candidatas con n-gramas antes del cálculo completo
- Resaltar los tramos coincidentes de una pareja
- Marcar parejas como revisadas sin borrarlas del informe

> ATRIBUTO DE CALIDAD — MODIFICABILIDAD
>
> **Se pide agregar una medida de similitud distinta de la distancia de edición.** Se agrega **creando una sola clase** que implementa la interfaz de métrica; no se toca el comparador, ni el controlador, ni la vista. Menos de treinta líneas, cronometradas y registradas en la bitácora.

> **El sistema no dictamina plagio.** Ordena por parecido y muestra evidencia; la decisión es del docente. La palabra «plagio» no aparece en la interfaz. La biblioteca nativa es **opcional**: `libdistancia.so` con el doble bucle, y cuenta como trabajo extra.

Juan Pablo Montoya Castro · Valentina Salazar Alzate

NOT FOUND `6 integrantes` `↔ Click + · 2 en común`

Gestor de memoria

Sistemas operativos · tema asignado

Un simulador de asignación de memoria: se definen bloques y solicitudes, y el sistema muestra cómo quedan la memoria y la fragmentación según la estrategia elegida. Es el otro ángulo de Sistemas operativos, distinto del planificador de Los Náufragos.

MICROSERVICIOS

- **`memoria-service`** — bloques, solicitudes y estado
- **`asignador-service`** — invoca la biblioteca nativa

FUNCIONALIDAD PRINCIPAL

- Definir el mapa de memoria y las solicitudes
- Aplicar primer ajuste, mejor ajuste y peor ajuste
- Ver la fragmentación interna y externa que deja cada una
- Comparar las tres estrategias sobre la misma carga

> BIBLIOTECA NATIVA — LIBMEMORIA.SO
>
> Las tres estrategias de asignación en C, sobre una estructura de bloques enlazados. Es el terreno natural de un lenguaje con punteros, y el contraste con hacerlo en Java es justamente el argumento arquitectural que deben defender.

> ATRIBUTO DE CALIDAD — USABILIDAD
>
> **Una solicitud de memoria falla porque no hay bloque suficiente.** El sistema **explica cuál fue la causa** —fragmentación externa, no falta de memoria total— en vez de responder un error genérico. Un simulador que no explica sus fallos no enseña nada.

> **Dos integrantes están además en Click +**, el equipo de Arquitectura que construye `libmemoria.so`. La biblioteca nativa es requisito del proyecto y se evalúa desde Arquitectura de Computadores, con independencia de cuántas personas cursen las dos asignaturas.

Shareltk Valentina Varón Castro · Andrey Mauricio Tabares Alférez · Rubén Darío Garzón Escobar · Robinson David Pérez Varela · Sebastián Galindo Mejía · Esteban Triviño Paredes

NEXUS `individual` `↔ Silicon Minds · sin personas en común`

Verificador de integridad

Matemáticas discretas · tema asignado

Un servicio que calcula y verifica huellas digitales de archivos usando aritmética modular, y detecta si un archivo cambió aunque sea en un byte. El alcance está reducido a propósito por ser un proyecto individual: dos servicios bien hechos, no cinco a medias.

MICROSERVICIOS

- **`huella-service`** — cálculo y verificación
- **`registro-service`** — historial de verificaciones

FUNCIONALIDAD PRINCIPAL

- Calcular la huella de un archivo subido
- Verificar un archivo contra una huella registrada
- Mostrar el historial de verificaciones
- Demostrar el efecto avalancha: un byte cambia, la huella cambia entera

> ATRIBUTO DE CALIDAD — DESEMPEÑO
>
> **Un usuario sube un archivo de 50 MB para verificar.** El servicio lo procesa **por bloques, sin superar 128 MB de memoria**, y responde en menos de 2 segundos. Demuéstrenlo midiendo la memoria del contenedor durante la subida: si crece proporcional al archivo, se está cargando entero y el escenario no se cumple.

> **Proyecto individual.** El alcance es menor y así se evalúa: no se compara contra el de un equipo de seis. La biblioteca nativa es opcional.

Juan Manuel Sarmiento Cubidez

DESARROLLOAPP `individual` `materia de otro semestre`

Simulador de enlace con pérdida

Modelamiento y simulación · toca además Comunicaciones y Redes 1 y Análisis de Algoritmos

Un enlace de red pierde paquetes con cierta probabilidad. Sobre ese único hecho salen los dos grandes tipos de algoritmo aleatorizado, y con un mapeo que no hay que forzar: la retransmisión hasta acuse de recibo —el protocolo ARQ que usa cualquier enlace real— **es** un algoritmo Las Vegas. Entrega el dato siempre correcto; lo que varía es cuánto tarda.

MICROSERVICIOS

- **`enlace-service`** — parámetros del enlace y escenarios guardados
- **`simulacion-service`** — motor de corridas y resultados

FUNCIONALIDAD PRINCIPAL

- Definir un enlace: probabilidad de pérdida, tamaño de paquete, retardo
- **Montecarlo** — estimar retransmisiones y la tasa de transferencia efectiva
- **Las Vegas** — simular ARQ: reenviar hasta que el paquete llegue bien
- Graficar la distribución de intentos y cómo el promedio se estabiliza
- Repetir una corrida exacta a partir de su semilla

> ATRIBUTO DE CALIDAD — TESTABILIDAD
>
> **Se repite una corrida guardando la misma semilla del generador aleatorio.** El resultado es **idéntico bit a bit al de la primera vez**, de modo que una prueba automática puede afirmar un valor exacto sobre un proceso aleatorio. Es el escenario a demostrar en la sustentación: sin semilla fija, este sistema no se puede probar.

> **Alcance por confirmar.** El alcance debe acordarse con la dirección del curso de Modelamiento y Simulación antes de iniciar, para que el mismo trabajo sirva en las dos materias. Ese acuerdo queda escrito, con fecha, en la primera entrada de la bitácora.

Francisco David Torres Aguirre

Cómo quedó el reparto

Dos equipos coinciden exactamente con su equipo de Arquitectura —Los Náufragos y Villanos— y responden por los dos frentes. En el resto la correspondencia es parcial, y por eso el proyecto se reparte por *entregable* y no por persona. Novadevs y Desarrolloapp no tienen equipo espejo: para ellos el componente nativo es opcional. Donde no hubo

materia común, el tema se asignó buscando la más cursada del grupo.

EQUIPO WEB 2	WEB 2	EQUIPO DE ARQUITECTURA	MATERIA ACOMPAÑANTE	ATRIBUTO EXIGIDO
Full-Stack Web	7	Low level software	Análisis numérico	**Desempeño**
Index Logic	4	Index logic	Arquitectura de computadores	**Desempeño**
Los Náufragos	2	**Los Náufragos**	Sistemas operativos	**Modificabilidad**
UCEVA Devs	5	Core Systems	Costos y presupuesto	**Testabilidad**
Not Found	6	Click +	Sistemas operativos	**Usabilidad**
Nexus	1	**Silicon Minds**	Matemáticas discretas	**Desempeño**
Villanos	2	**Villanos**	Matemáticas discretas	**Disponibilidad**
Novadevs	2	—	Matemáticas discretas	**Modificabilidad**
Desarrolloapp	1	—	Modelamiento y simulación	**Testabilidad**

Los alcances están calibrados al tamaño de cada equipo: un proyecto individual no se evalúa con la misma vara que uno de seis. Lo que sí se exige igual a todos es la justificación del diseño — la bitácora de decisiones pesa tanto como el sistema funcionando, porque un sistema que funciona sin que nadie sepa por qué está hecho así no se puede mantener ni defender.